Universidad del Valle
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación



Guía:
Poliformismo





El siguiente documento ha sido elaborado tomando fragmentos de documentos previamente publicados, así como imágenes y ejercicios, sobre los cuales se han realizado traducciones libres y adaptaciones con el propósito de desarrollar el material para el curso de Fundamentos de Programación Orientada a Objetos de la Universidad del Valle. Se ha respetado la autoría original de los materiales, y se han proporcionado las referencias correspondientes dentro del documento. El uso de este material se restringe estrictamente al marco académico y formativo de los estudiantes que cursen la asignatura, y no tiene fines comerciales. Queda prohibida su copia o distribución fuera del contexto de dicho curso. *Una primera versión con los estudiantes de doctorado: Giovanny Hidalgo, y José Llanos.*

Polimorfismo

El polimorfismo es uno de los conceptos clave de la programación orientada a objetos (POO) [1]. La palabra polimorfismo viene del griego, donde *"poli"* significa muchos y *"morfos"* formas, aunque en el contexto de programación, "*morfos*" se refiere más a acciones. En términos prácticos, el polimorfismo permite que los objetos derivados de una misma clase base respondan de manera distinta a un mismo llamado de un método, dependiendo de su propia implementación específica. Esto fomenta el diseño de sistemas más flexibles, reutilizables y fáciles de mantener, ya que la interacción con los objetos se realiza de manera genérica, mientras que el comportamiento específico queda encapsulado en la programación específica en cada clase y cada método [2].

¿Cómo funciona el polimorfismo?

Consideremos una interfaz común, por ejemplo en los electrodomésticos, el botón de encender. El botón está en diferentes aparatos: lámparas, televisores, ventiladores, entre otros. Aunque todos tienen un botón de encendido, la acción que realizan es distinta según el tipo de electrodoméstico. La lámpara emite luz, la televisión muestra imágenes y el ventilador comienza a girar las aspas. Desde la perspectiva del usuario, solo se necesita conocer el concepto general de "encender", sin preocuparse por los detalles específicos o internos de cada dispositivo. Esto es exactamente lo que hace el polimorfismo en programación: ofrecer una forma general de interactuar con objetos mientras se delega el comportamiento específico a cada implementación particular.

El polimorfismo no es exclusivo de la programación. Lo podemos ver en la naturaleza, donde un mismo ser vivo cambia de forma y comportamiento. Un ser humano, por ejemplo, pasa por distintas etapas de desarrollo, cada etapa con sus propias características y comportamientos. Veamos el caso de la respiración, en el acto de "respirar" un feto respira a través del cordón



umbilical, mientras que un adulto lo hace a través de los pulmones. La acción es la misma, pero la implementación es diferente, en este caso propia o específica de cada etapa.

En el reino animal las diferentes especies comparten un comportamiento común, sin embargo cada especie lo ejecuta de forma única, según su adaptación al entorno. Por ejemplo, todos los mamíferos respiran, pero la acción de respirar es diferente entre mamíferos. Veamos el caso de la ballena y el ratón: las dos especies respiran oxígeno, pero la ballena, como mamífero marino, está adaptada para respirar a través de un espiráculo mientras permanece sumergida en el agua. Por otro lado, los ratones están diseñados para respirar mediante las fosas nasales y la boca. Aunque los dos comparten el comportamiento de "respirar", la manera de llevarlo a cabo depende de su tipo.

Este ejemplo refleja cómo el polimorfismo permite que una acción compartida, como respirar, se adapte a las necesidades y características específicas de cada tipo. En programación orientada a objetos, la clase base llamada "Mamífero" define un método "respirar", el cual es sobreescrito por las clases derivadas "Ballena" y "Humano", cada una implementando su propia versión del método “respirar”. Esta diversidad en la implementación de un comportamiento común es lo que define al polimorfismo [3]. Así se muestra la flexibilidad del concepto y también su capacidad para modelar sistemas complejos basados en comportamientos compartidos pero especializados [4].

El concepto del **polimorfismo** puede extenderse fácilmente al ejemplo de un ser humano que respira en condiciones especiales, como dentro del agua utilizando un tanque de oxígeno. En términos de programación orientada a objetos, podemos modelar esto con una jerarquía de clases que represente el comportamiento genérico de los mamíferos y las particularidades del ser humano en diferentes contextos. Así, imaginemos una **clase base** llamada `Mamifero`, que define un método genérico de respirar, con un parámetro que indica el medio donde ocurre la respiración (por ejemplo, aire o agua). Luego, tenemos una **clase derivada** llamada `Humano`, que hereda de `Mamifero` y sobrescribe el método `respirar` para manejar contextos específicos como bucear con un tanque de oxígeno. El parámetro del método permite que el comportamiento de la respiración sea adaptable al contexto, por medio de la **reescritura de métodos** [5].

Explicación Detallada del Polimorfismo

1. **Clase Base (`Mamífero`)**: Representa el comportamiento genérico de todos los mamíferos. Tiene un método `respirar(medio)` que describe cómo el mamífero respira en un medio determinado. Por defecto, el método podría asumir que el medio es el aire.



2. **Clase Derivada (Humano)**: Hereda de `Mamífero` y sobrescribe el método `respirar`. Cuando el parámetro `medio` es `"agua"`, el humano utiliza un tanque de oxígeno para respirar, adaptando el comportamiento del método.
3. **Polimorfismo**: Desde el punto de vista genérico, ambos objetos (`Mamífero` y `Humano`) tienen el método `respirar`, pero el comportamiento exacto depende del tipo específico de objeto y del valor del parámetro, a través de la **reescritura del método.**
4. Nota: **Polimorfismo**:s**e puede implementar solo como una sobreescritura de métodos sin la necesidad de la relación de herencia.**

Este ejemplo ilustra cómo diferentes tipos de mamíferos comparten una interfaz común (`respirar`), mientras que las clases derivadas (como `Humano`) pueden agregar comportamientos específicos. El parámetro en el método ayuda a manejar casos particulares sin necesidad de cambiar la clase base, lo que refuerza los principios de extensibilidad y reutilización del código. Así, el polimorfismo en programación puede ser entendido como una mismo llamado a una acción que puede realizarse de diversas formas según el "estado" o "tipo" del objeto [5].

En programación orientada a objetos, el polimorfismo se inspira en el principio de especialización. Imagina que tienes un grupo de objetos que comparten una misma "especie" (una clase base), pero cada especie tiene su propio comportamiento único. Al igual que en biología, esta capacidad de adaptarse a diferentes formas permite que el software sea flexible, manteniendo al mismo tiempo una base común que hace que todo funcione como un sistema unificado [6].

En general, el polimorfismo nos permite modelar la diversidad de comportamientos que comparten una misma abstracción. Por ejemplo, todos los animales se mueven, pero lo hacen de formas distintas: un perro corre, una ballena nada. Esto demuestra que una misma acción puede tener múltiples implementaciones según el objeto que la ejecute. En otras palabras, el polimorfismo permite que un único punto de acceso, como el nombre de un método, pueda activar distintas acciones dependiendo de cómo se implemente en cada caso. Por esta razón, el polimorfismo a menudo se describe como "una interfaz, múltiples implementaciones" [5]. Esto hace que las clases puedan compartir un método común mientras que sus comportamientos específicos correspondan a las necesidades.

El polimorfismo permite que los objetos se llamen de forma general, aunque internamente se comportan de acuerdo a su especificidad. Así, el polimorfismo, se refiere al hecho de que varios objetos de diferentes clases, que comporten una clase base, se pueden comportar de manera particular, sin tener que conocer la clase exacta de la que deriva. De igual manera, permite a un objeto realizar una acción de diferentes formas, utilizando el mismo mensaje o nombre del método [5].



Hay distintas formas de implementar el polimorfismo en C++. El polimorfismo se emplea cuando realizamos **sobrescritura de métodos**, lo que nos permite redefinir el comportamiento de un método dentro de una misma clase, sin necesidad de recurrir a la herencia [7]. A diferencia de la sobrecarga, donde el mismo método tiene diferentes implementaciones basadas en los parámetros de entrada, la sobrescritura consiste en modificar el comportamiento de un método ya definido en la misma clase, manteniendo su firma pero cambiando su lógica interna. Este enfoque es útil cuando queremos que un método se adapte a diferentes situaciones dentro de la misma clase, permitiendo una mayor flexibilidad y reutilización del código, ya que los objetos pueden comportarse de manera distinta según el contexto, incluso dentro de la misma clase. Esto facilita la escritura de código más limpio y modular, evitando la duplicación y mejorando la mantenibilidad. El polimorfismo en C++ se consigue usando sobreescritura de métodos [7].

Por otra parte, el polimorfismo se pone en práctica cuando una clase base define funciones virtuales, y una o varias clases derivadas heredan de ella. Permitiendo que un mismo mensaje, enviado a objetos de diferentes clases, provoque respuestas distintas [5]. Al marcar un método como virtual en la clase base, le estamos dando la capacidad de ser sobrescrita por las clases derivadas, permitiendo que cada clase tenga su propia implementación de esa función. De esta manera, aunque se invoque el mismo método en objetos de diferentes tipos, cada uno ejecutará el código correspondiente a su clase, creando un comportamiento dinámico y flexible.

Esta capacidad se relaciona estrechamente con el concepto de herencia, ya que las clases derivadas heredan los comportamientos de la clase base pero pueden modificarlos según sus necesidades específicas. El polimorfismo en C++ se consigue usando herencia con funciones virtuales y un puntero a clase base con el que apuntar a distintos objetos derivados. Con la definición de una función virtual, es obligatorio declarar el destructor como virtual [5]..

El polimorfismo facilita la construcción de programas más concisos y eficientes, ya que permite escribir código más corto y modular sin perder flexibilidad [1]. Al permitir que un mismo método sea interpretado de diferentes maneras según el objeto que lo invoque, el polimorfismo elimina la necesidad de escribir múltiples versiones de un mismo comportamiento para distintos tipos de objetos. Esto no solo reduce la cantidad de código repetitivo, sino que también facilita la modificación y adaptación del programa. Si es necesario cambiar la implementación de un comportamiento, solo se ajusta en un solo lugar, y todos los objetos relacionados con ese comportamiento se verán automáticamente actualizados sin necesidad de realizar cambios adicionales en todo el sistema [6]. Gracias al polimorfismo, podemos definir interfaces comunes para diferentes clases, lo que facilita la creación de módulos independientes que interactúan entre sí de manera flexible. Esto significa que las partes del código que no están directamente relacionadas pueden mantenerse separadas, lo que mejora la legibilidad y el mantenimiento del software.

Polimorfismo estático y dinámico

El polimorfismo estático se lleva a cabo en tiempo de compilación y se programa a través de overload o sobrecarga de métodos [5]. En C++ es posible sobreescribir un método en distintas versiones, ellas deben modificar sus parámetros o salidas y el compilador asigna específicamente la apropiada para cada caso, por este motivo se conoce como polimorfismo en tiempo de compilación [5]. La sobreescritura de métodos permite mayor modularidad y especificidad en los módulos de los programas. De igual manera, cada método sobreescrito puede llevar una implementación particular tal como se explicó en la sección anterior.

En este caso particular, el polimorfismo se refiere a un método que tienen el mismo nombre, pero se distinguen por los argumentos, ya sean de entrada o de salida, por ejemplo, mover(); mover(int x) y mover(int x, int y) sería una sobreescritura del método mover.

El polimorfismo dinámico se lleva a cabo en tiempo de ejecución y se programa a través de un puntero a una clase Base y un apuntador a sus clases Derivadas, posteriormente se llama a los métodos que se requieran, sin necesitar saber exactamente a cuál objeto pertenecen [5]. En C++ es necesario implementar métodos virtuales y en ellos las distintas versiones o comportamientos para cada una de las clases Derivadas, cada método implementa la modificación de sus comportamientos y en tiempo de ejecución estos se asignan dinámicamente de acuerdo al puntero y la tabla de función virtual (v-table), la cual almacena los punteros de cada objeto de las clases Derivadas. Por este motivo se conoce como polimorfismo en tiempo de ejecución [5].

Funciones virtuales o lo que podrían llamarse métodos virtuales, cada clase tiene una serie de funciones o métodos disponibles, los cuales se enlazan o se ligan normalmente en tiempo de compilación, a esto se conoce como ligadura de funciones [7]. Sin embargo, para la implementación del polimorfismo la ligadura de funciones o métodos se puede hacer en tiempo de ejecución, para ello en C++ se utiliza la palabra reservada ***virtual*** delante de la función [7]. Esto permite que la decisión de cuál método ligar: *i*) el de la clase Base o *ii*) el de la clase Derivada, se tome en tiempo de ejecución. Considere el siguiente ejemplo, la clase Base motor y las clases Derivadas Motor eléctrico y Motor a gas, ver el diagrama de clases en la Figura 1. (ejemplo tomado de [8]).

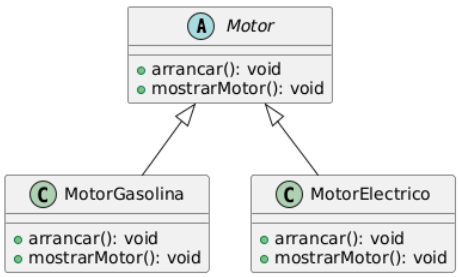


Figura 1. Diagrama de clases tipos de motores



Para implementar el polimorfismo es necesario, definir los métodos como virtuales, indicando al compilador que genere código para el tipo de objeto en tiempo de ejecución y que espere para elegir la función o el método apropiado. Ver siguiente recuadro.

```
1   #define MOTOR_H
2
3   #include <string>
4
5   class Motor {
6   public:
7       // Constructor virtual por defecto
8       Motor() = default;
9
10      // Destructor virtual para garantizar la destrucción correcta
    de objetos derivados
11      virtual ~Motor() = default;
12
13      // Método puro (virtual) para arrancar el motor
14      virtual void arrancar() const = 0;
15
16      // Método puro (virtual) para mostrar el tipo de motor
17      virtual std::string mostrarMotor() const = 0;
18  };
19
20  #endif // MOTOR_H
```

Recuadro 1. Definición de la clase Motor

Esto se debe definir de forma similar en las clases Derivadas, la misma definición se utiliza en la clase motor eléctrico, ver Recuadro 2. Finalmente, se hace la llamada del objeto en tiempo de ejecución mediante un puntero, ver Recuadro 3.

```cpp
#ifndef MOTOR_GASOLINA_H
#define MOTOR_GASOLINA_H

#include "Motor.h"
#include <iostream>
#include <string>

class MotorGasolina : public Motor {
public:
    // Constructor por defecto
    MotorGasolina() = default;

    // Destructor
    ~MotorGasolina() override = default;

    // Implementación del método arrancar
    void arrancar() const override;
    // Implementación del método mostrarMotor
    std::string mostrarMotor() const override;

#endif // MOTOR_GASOLINA_H
```

Recuadro 2. Definición de la clase Motor Gasolina

```cpp
Motor* motor = new MotorGasolina(); // El puntero motor apunta a un objeto MotorGasolina
motor->arrancar();                  // Llama al método arrancar() de MotorGasolina
delete motor;                       // Liberamos la memoria

motor = new MotorElectrico();       // Ahora apunta a un objeto MotorElectrico
motor->arrancar();                  // Llama al método arrancar() de MotorElectrico
delete motor;                       // Liberamos la memoria
```

Recuadro 3. Main uso de polimorfismo

Esto es aplicable entonces a todos los objetos de las clases derivadas.

```
Usando un Motor de Gasolina:
El motor de gasolina está arrancando...
Consumo estimado: 0.08 litros por km

Usando un Motor Eléctrico:
El motor eléctrico está arrancando...
Consumo estimado: 150 watts por km
```

Recuadro 4. Salida esperada aplicando polimorfismo

En la Figura 2. se describe como está formado el objeto Motor a Gasolina, el cual hereda del objeto Motor, en su parte de Motor tiene una v-table, la cual contendrá los punteros a las funciones de las llamadas a la función o método específico de la clase Derivada. En el ejemplo se ha creado un puntero a Motor a Gasolina, con lo cual en tiempo de ejecución el programa llama arrancar, y muestra información del motor, en el caso de que tuviéramos otra implementación específica para motor hibrido, entonces en el v-table de Motor, tendríamos el apuntador a la acción o método de la motor híbrido con su propia implementación para arrancar y mostrar información.

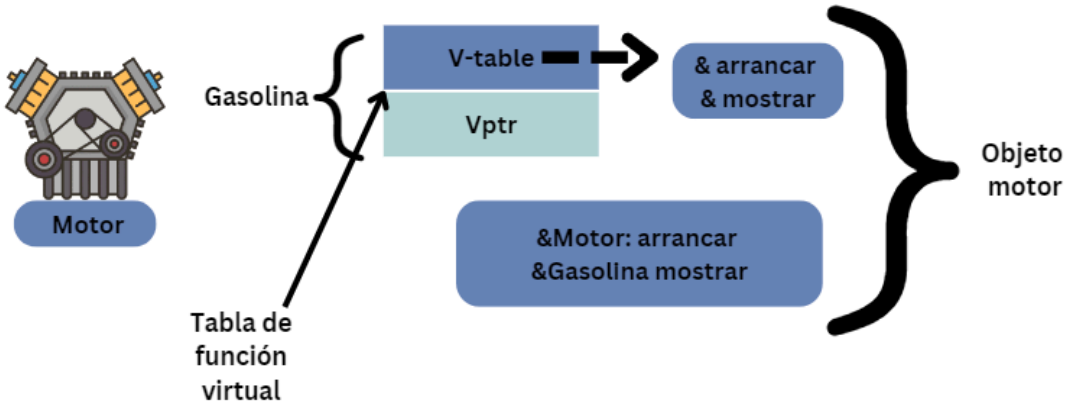


Figura 2. VTAble en C++ fuente: figura tomada de: https://www.youtube.com/watch?v=HY-cTsxajTk

Funciones virtuales puras. Se declaran en la clase base sin una implementación concreta, usando =0 A estas funciones que no tienen código, se las llama funciones virtuales puras o clases abstractas [7]. Las clases que contienen al menos una función virtual pura se las llama clases abstractas. De ellas no se pueden instanciar objetos, debido a la falta del código de las

funciones virtuales puras. Lo único que se puede hacer con una clase abstracta es heredar de ella e implementar en la clase derivada las funciones faltantes [7].

El polimorfismo permite ejecutar la función correcta automáticamente o lo que se llama DISPATCH. El ***Dispatch simple*** consiste en llamar a un método perteneciente a la clase base, y el compilador enruta automáticamente la función correcta en la clase Derivada [7]. Cuando hay polimorfismo, el compilador de C++ decide cual de todos los métodos ejecutar. Continuando con el ejemplo podemos reescribir el siguiente código, ver Recuadro 5.

Motor a gasolina

```
1  void mostrarMotor() const override {
2  std::cout << "Información del Motor de Gasolina:" << std::endl;
   std::cout << "- Número de pistones: 4" << std::endl;
3  std::cout << "- Torque: 250 Nm" << std::endl;
4  std::cout << "- Potencia: 150 HP" << std::endl;
5  }
```

Motor eléctrico

```
1  void mostrarMotor() const override {
2  std::cout << "Información del Motor Eléctrico:" << std::endl;
   std::cout << "- Potencia nominal: 200 kW" << std::endl;
3  std::cout << "- Torque instantáneo: 400 Nm" << std::endl; std::
   << "- Eficiencia energética: 95%" << std::endl;
4   }
```

```
1  int main()
2  {
3   vector<Motor *>
   Motores;
4
   Motores.push_back(ne
   w CGasMotor);
5
   Motores.push_back(ne
   w CElectircMotor);
6
   Motores.push_back(ne
   w CGasMotor);
7   for(int cual=0;
   cual<3; cual++)
8   Motores[cual]-
   >MostrarMotor();
9  }
```

Recuadro 5 Ejemplo de implementación de polimorfismo con vectores

Dispatch doble consiste en que la función que se ejecuta depende del objeto que recibe la orden y del argumento de entrada que hay en la orden [7]. Es posible programar dispatch triple y a esto se le conoce como polimorfismo múltiple y se logra teniendo varios punteros en la misma sentencia, apuntando a sus respectivas clases base de sus árboles de herencia. El dispatch doble requiere 2 punteros,ver ejemplo en el Recuadro 6.




```h
#ifndef MOTOR_H
#define MOTOR_H

#include <iostream>
#include <string>

class DiagnosticSystem; // Declaración adelantada

class Motor {
public:
    virtual ~Motor() = default;

    virtual std::string mostrarMotor() const = 0;
    virtual void aceptarDiagnostico(DiagnosticSystem& diagnostico) const = 0; // Método para dispatch doble
};

#endif // MOTOR_H
```

```h
#ifndef MOTOR_GASOLINA_H
#define MOTOR_GASOLINA_H

#include "Motor.h"
#include "DiagnosticSystem.h"

class MotorGasolina : public Motor {
public:
    std::string mostrarMotor() const override {
        return "Motor de Gasolina";
    }

    void aceptarDiagnostico(DiagnosticSystem& diagnostico) const override {
        diagnostico.diagnosticar(*this); // Dispatch doble
    }
};

#endif // MOTOR_GASOLINA_H
```




```cpp
#ifndef MOTOR_ELECTRICO_H
#define MOTOR_ELECTRICO_H

#include "DiagnosticSystem.h"
#include "Motor.h"

class MotorElectrico : public Motor {
public:
  std::string mostrarMotor() const override { return "Motor Eléctrico"; }

  void aceptarDiagnostico(DiagnosticSystem &diagnostico) const override {
    diagnostico.diagnosticar(*this); // Dispatch doble
  }
};

#endif // MOTOR_ELECTRICO_H
```




```cpp
#ifndef DIAGNOSTIC_SYSTEM_H
#define DIAGNOSTIC_SYSTEM_H

#include <iostream>
#include "MotorGasolina.h"
#include "MotorElectrico.h"

class DiagnosticSystem {
public:
    void diagnosticar(const MotorGasolina& motor) const {
        std::cout << "Diagnóstico para " << motor.mostrarMotor() << ":" << std::endl;
        std::cout << "- Revisar bujías." << std::endl;
        std::cout << "- Comprobar nivel de aceite." << std::endl;
    }

    void diagnosticar(const MotorElectrico& motor) const {
        std::cout << "Diagnóstico para " << motor.mostrarMotor() << ":" << std::endl;
        std::cout << "- Verificar nivel de carga de la batería." << std::endl;
        std::cout << "- Comprobar aislamiento de los cables." << std::endl;
    }
};
```

```cpp
#include <iostream>
#include <memory>
#include "Motor.h"
#include "MotorGasolina.h"
#include "MotorElectrico.h"
#include "DiagnosticSystem.h"

int main() {
    DiagnosticSystem diagnostico;

    // Crear un motor de gasolina y ejecutar el diagnóstico
    std::unique_ptr<Motor> motorGasolina = std::make_unique<MotorGasolina>();
    std::cout << "Usando un " << motorGasolina->mostrarMotor() << " para diagnóstico:" << std::endl;
    motorGasolina->aceptarDiagnostico(diagnostico);

    std::cout << std::endl;

    // Crear un motor eléctrico y ejecutar el diagnóstico
    std::unique_ptr<Motor> motorElectrico = std::make_unique<MotorElectrico>();
    std::cout << "Usando un " << motorElectrico->mostrarMotor() << " para diagnóstico:" << std::endl;
    motorElectrico->aceptarDiagnostico(diagnostico);

    return 0;
}
```

Recuadro 6. ejemplo de dispatch doble

Referencias

[1] J. A. Villalobos S., *Fundamentos de programación: aprendizaje activo basado en casos : un enfoque moderno usando Java, UML, objetos y eclipse*. Colombia: Pearson, 2006.

[2] R. C. Martin, J. Grenning, S. Brown, y K. Henney, *Clean Architecture: a craftsman's guide to software structure and design*. en Robert C. Martin series. Boston Columbus Indianapolis New York San Francisco Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo: Prentice Hall, 2018.



[3] D. West, *Object thinking*. en Microsoft professional. Redmond, WA: Microsoft, 2004.
[4] M. A. Weisfeld, *The object-oriented thought process*, 3rd ed. en Developer's library. Upper Saddle River, NJ: Addison-Wesley, 2009.
[5] E. Balagurusamy, *Programación orientada a objetos con C++*, 4ª ed. Madrid: McGraw-Hill, 2007.
[6] *Lógica y programación orientada a objetos: un enfoque basado en problemas*. Medellín: Tecnologico de Antioquia, 2009.
[7] B. Cyganek, *Introduction to programming with C++ for engineers*. Chichester, West Sussex, UK ; Hoboken, NJ, USA: Wiley-IEEE Press, 2020.
[8] «Ejemplo polimorfismo», C++ polimorfismo. Accedido: 1 de enero de 2025. [En línea]. Disponible en: https://slidetodoc.com/polimorfismo-y-funciones-virtuales-agustn-j-gonzlez-versin/